NEXASTORE

AI Machines • Developer Workstations • Next-Generation Computing



RAG + EMBEDDINGS

Hardware for RAG and Embedding Workloads

Size developer machines for retrieval pipelines

NexaStore Knowledge Series • 2026 Edition

1. RAG Is More Than LLM Inference

A Retrieval-Augmented Generation system combines document ingestion, text extraction, chunking, embedding generation, vector indexing, retrieval, reranking and LLM generation. Hardware needs therefore depend on which parts run locally and which are consumed as managed cloud services.

RAG stage	Main hardware pressure	Notes
Document extraction	CPU, RAM, storage	PDF parsing and OCR can be CPU/memory intensive.
Chunking / preprocessing	CPU, RAM	Usually does not require a discrete GPU.
Embedding generation	CPU/GPU/NPU depending on model	Batch size and local-vs-cloud choice matter.
Vector indexing	RAM, storage, CPU	Index size grows with chunk count and embedding dimension.
Retrieval	RAM, storage latency	Vector search benefits from adequate memory and fast storage.
Reranking	CPU/GPU depending on model	Local cross-encoders can add compute cost.
Generation	Cloud or local accelerator	Local LLM generation may dominate GPU requirements.

2. Developer Machine Profiles

Profile	RAM	GPU	Storage	Typical architecture
Cloud-first RAG	32 GB	Optional	1 TB	Cloud embeddings + cloud LLM + loca app/database
Hybrid RAG developer	64 GB	8-12 GB useful	2 TB	Local vector DB + some local embeddings/inference
Local AI/RAG lab	64-128 GB	16-24+ GB	2-4 TB	Local embeddings, reranker, LLM and vector DB
Large corpus lab	128 GB+	Workload dependent	4 TB+	Large indexes, datasets and multiple lo AI services

3. Embedding Workloads

Scenario	Hardware direction
Cloud embedding API	GPU not required locally; prioritize network, CPU and application memory.
Small local embedding model	CPU can be adequate; GPU can improve throughput.
Large ingestion batch	More CPU cores, RAM and GPU acceleration can shorten ingestion time.
Continuous ingestion	Balance throughput, cost and latency; consider background workers and managed services.
Multiple embedding models	Plan RAM/VRAM and model cache/storage for concurrent model residency.

4. Vector Store Considerations

Factor	Effect
Chunk count	More chunks increase index size and retrieval work.
Embedding dimension	Higher-dimensional vectors consume more storage and memory.
Metadata	Useful for filtering; also adds index/storage overhead.
Hybrid search	Adds keyword/full-text indexing alongside vector search.
Concurrency	More simultaneous queries increase CPU, memory and I/O requirements.

5. Example NexaStore RAG Development Machine

Component	Recommended development direction
CPU	Modern multi-core processor for extraction, chunking, builds and containers
RAM	64 GB for comfortable multi-service local development
GPU	12-16 GB VRAM if running embeddings/reranking/local LLMs frequently
Storage	2 TB NVMe or more for PDFs, indexes, models, containers and datasets
Network	Reliable high-speed connection for cloud model/search services
Display / ports	Developer ergonomics; multiple displays and dock support where needed

6. Architecture Rule

Do not overbuy GPU hardware merely because the application uses RAG. If embeddings, search and LLM inference are all managed cloud services, a strong developer laptop may be sufficient. Local accelerators become important when the architecture deliberately moves embedding, reranking or generation workloads onto the developer machine.

7. Agent Recommendation Use

For questions such as 'Which machine should I buy for building RAG systems?', the agent should first establish whether models and search run locally or in the cloud. It can then combine this knowledge with structured catalog facts such as RAM, GPU VRAM, SSD capacity, price and availability.

Educational notice

This document is part of the fictional NexaStore educational case study.